

Time: 2026-08-26 21:14

Author: Trần Xuân Bách

Status: #seed

Tags: Deep Learning

Perceptron

Tuyến tính (Linear): Các yếu tố hoạt động độc lập

Một mô hình tuyến tính hoạt động dựa trên nguyên tắc cộng dồn. Bạn có thể tưởng tượng nó giống như cách tính tiền khi đi siêu thị.

Giả sử bạn mua x_1 quả táo (giá 10 ngàn) và x_2 quả chuối (giá 5 ngàn). Tổng tiền $= 10 \cdot x_1 + 5 \cdot x_2$.

Đặc điểm của mô hình này là: Nếu bạn bỏ thêm 1 quả táo vào giỏ (x_1 tăng thêm 1), tổng tiền của bạn **luôn luôn** tăng thêm đúng 10 ngàn. Việc bạn đang có 0 quả chuối hay 100 quả chuối (x_2) không hề làm thay đổi giá trị của quả táo. Táo và chuối đóng góp vào tổng tiền một cách hoàn toàn độc lập, không liên quan gì đến nhau.

Về mặt hình học, vì mọi thứ tăng giảm đều đặn, ranh giới để mô hình ra quyết định luôn là một đường thẳng tắp (không cong, không gấp khúc).

Phi tuyến tính (Non-linear) và Bài toán XOR: Sự tương tác

Phi tuyến xuất hiện khi quy luật độc lập ở trên bị phá vỡ: tác dụng của yếu tố này phụ thuộc vào yếu tố kia.

Bài toán XOR nghe có vẻ trừu tượng, nhưng nó chính là **cơ chế của hệ thống đèn cầu thang** trong nhà bạn (hai công tắc ở hai đầu cầu thang cùng điều khiển một bóng đèn). Quy luật của đèn cầu thang (XOR) là:

- Nếu cả 2 công tắc cùng gạt xuống (0,0) hoặc cùng gạt lên (1,1): Đèn tắt (0).
- Nếu một cái gạt lên, một cái gạt xuống (1,0 hoặc 0,1): Đèn sáng (1).

Bây giờ, hãy phân tích tác động của công tắc số 1 (x_1) đối với bóng đèn:

- Nếu công tắc 2 đang ở vị trí TẮT ($x_2=0$): Gạt công tắc 1 lên sẽ làm đèn chuyển từ Tắt sang Sáng. Tức là x_1 có tác động **tích cực** (làm tăng giá trị từ 0 lên 1).
- Nếu công tắc 2 đang ở vị trí BẬT ($x_2=1$): Gạt công tắc 1 lên sẽ làm đèn chuyển từ Sáng sang Tắt. Tức là x_1 lúc này lại có tác động **tiêu cực** (làm giảm giá trị từ 1 về 0).

Đây chính là điểm "gãy" của mô hình tuyến tính. Mô hình tuyến tính bắt buộc một biến số phải có một tác động cố định (như quả táo luôn làm tăng 10 ngàn). Nó không thể hiểu được logic: *"Lúc thì thêm x_1 làm kết quả tăng, lúc thì thêm x_1 lại làm kết quả giảm, tùy thuộc vào việc x_2 đang là gì"*.

Sự đảo chiều tác động này chính là sự tương tác. Hàm XOR là bài toán đơn giản nhất chứng minh được sự tồn tại của các tương tác này, và chứng minh rằng mô hình tuyến tính hoàn toàn bất lực trước chúng.

Hidden Layer

Tưởng tượng một căn phòng có hai công tắc đèn. Bạn muốn dạy máy quy tắc này: **đèn sáng khi đúng một công tắc được bật**. Cả hai tắt thì tối, cả hai bật cũng tối.

Đó là XOR. Bốn tình huống:

- Cả hai tắt → tối
- Chỉ công tắc A bật → sáng
- Chỉ công tắc B bật → sáng
- Cả hai bật → tối

Không có gì bí ẩn ở đây. Vấn đề nằm ở chỗ máy được phép suy nghĩ theo kiểu nào

Mô hình tuyến tính suy nghĩ như một bảng chấm điểm

Một mô hình tuyến tính hoạt động đúng như chấm điểm thi. Mỗi đặc trưng được gán một số điểm cố định. Cộng tất cả lại. So với một mức chuẩn. Trên chuẩn thì "có", dưới chuẩn thì "không".

Hai tính chất của cách suy nghĩ này khiến nó chết trước XOR.

Thứ nhất, điểm của mỗi đặc trưng là cố định. Công tắc A bật thì luôn cộng đúng chừng đó điểm, bất kể công tắc B đang ở trạng thái nào. Nhưng XOR đòi hỏi ngược lại: khi B đang tắt, bật A phải làm đèn sáng lên; khi B đang bật, bật A phải làm đèn tối đi. Cùng một hành động, tác dụng ngược nhau tùy hoàn cảnh. Bảng chấm điểm không có chỗ để ghi điều đó.

Thứ hai, chỉ có một mức chuẩn. Đây là cách nhìn sắc bén hơn. Vì mô hình tuyến tính chỉ biết cộng, thứ duy nhất nó thực sự đo được ở đây là **có bao nhiêu công tắc đang bật**: không, một, hay hai. Và câu trả lời đúng là: không → tối, một → sáng, hai → tối.

Một mức chuẩn duy nhất chỉ chia được trục thành hai nửa: "từ đây trở lên" hoặc "từ đây trở xuống". Nó không bao giờ khoanh được một khoảng ở giữa. Còn XOR đòi hỏi đúng cái đó: **nhều hơn không, nhưng ít hơn hai**.

Đây là lý do XOR không giải được, phát biểu mà không cần một ký hiệu toán nào.

Đặt câu hỏi trung gian

Cách thoát ra rất đơn giản. Nếu bạn cần hai mức chuẩn mà mỗi đơn vị chỉ có một, thì hãy dùng **hai đơn vị**, rồi kết hợp câu trả lời của chúng.

Hidden layer là một tầng gồm những đơn vị như thế. Mỗi đơn vị nhìn vào đầu vào và trả lời **một câu hỏi con**. Tầng cuối không nhìn đầu vào gốc nữa — nó chỉ nhìn các câu trả lời.

Với XOR, hai câu hỏi là:

- **Câu hỏi 1:** có bao nhiêu công tắc đang bật?
- **Câu hỏi 2:** vượt quá một công tắc bao nhiêu?

Bảng trả lời:

Tình huống	Câu 1	Câu 2	Kết luận: câu 1 trừ hai lần câu 2
Cả hai tắt	0	0	$0 \rightarrow$ tối
Chỉ A bật	1	0	$1 \rightarrow$ sáng
Chỉ B bật	1	0	$1 \rightarrow$ sáng
Cả hai bật	2	1	$2 - 2 = 0 \rightarrow$ tối

Bốn đáp án đúng cả bốn. Và hãy để ý: tầng cuối làm một việc hoàn toàn tuyến tính — nó chỉ cộng và trừ hai con số. Sự thông minh nằm ở chỗ **câu hỏi được đặt ra như thế nào**, không nằm ở phép kết hợp.

Chỗ nào là "phi tuyến"?

Câu hỏi 2 có một điều kiện ngầm mà tôi chưa nói rõ: **không được trả lời số âm**. Nếu không có công tắc nào bật, câu trả lời phải là 0, chứ không phải -1 .

Đó chính xác là việc ReLU làm. Nó cắt bỏ phần âm, giữ nguyên phần dương. Nghe như một chi tiết vụn vặt, nhưng nó là toàn bộ phép màu.

Thử bỏ nó đi. Khi cả hai công tắc tắt, câu hỏi 2 sẽ trả lời -1 thay vì 0. Kết luận trở thành $0 - 2 \times (-1) = 2$. Đèn sáng gấp đôi, trong khi đáp án đúng là tối. Ba trường hợp kia vẫn ổn; chỉ riêng trường hợp này gây — và mạng lại trở về đúng một mô hình tuyến tính vô dụng như cũ.

Nói cách khác: **nếu mọi tầng đều chỉ cộng và nhân, thì chòng bao nhiêu tầng cũng vẫn quy về một tầng**. Cộng rồi lại cộng thì vẫn là cộng. Phải có một thao tác nào đó "gãy khúc" — cắt bỏ phần âm là thao tác gãy khúc rẻ nhất mà người ta tìm ra.

Việc thật sự mà hidden layer làm

Nhìn lại bảng trên, để ý hai dòng giữa: "chỉ A bật" và "chỉ B bật" là hai tình huống **khác nhau ở đầu vào**, nhưng sau khi qua tầng câu hỏi, chúng cho **cùng một cặp câu trả lời** (1 và 0). Chúng trở thành không phân biệt được.

Đó là mô tả chính xác nhất về công việc của hidden layer: **làm cho những thứ đáng được đối xử giống nhau trở thành thực sự giống nhau**. Khi hai văn bản khác hẳn về câu chữ nhưng cùng nói về bóng đá, một hidden layer tốt sẽ đẩy chúng về gần nhau trong không gian câu trả lời, để tầng cuối chỉ còn việc kẻ một lát cắt.

Và điểm cuối cùng, quan trọng nhất: trong ví dụ XOR tôi đã tự nghĩ ra hai câu hỏi đó. Trong thực tế **không ai nghĩ hộ**. Huấn luyện mạng chính là quá trình để nó tự tìm ra nên hỏi những câu gì. Với bộ tin tức của bạn, một hidden unit có thể tự học thành câu hỏi "văn bản này đậm chất thể thao tới mức nào", một cái khác thành "có mùi tài chính không" — nhưng không ai viết ra những câu hỏi ấy, và chúng cũng thường không gọn gàng để đặt tên như vậy.

Đó là khác biệt giữa TF-IDF và hidden layer. TF-IDF là bộ câu hỏi do con người soạn sẵn: "từ này xuất hiện bao nhiêu lần". Hidden layer là bộ câu hỏi được tìm ra từ chính dữ liệu và chính nhiệm vụ.

SVM và Kernel

Đầu tiên, cần làm rõ một chút về câu chữ để tránh sự hiểu lầm: Bài viết trước không nói rằng SVM thất bại trước bài toán tuyến tính, mà ý của nó là: Bản thân SVM luôn luôn là một cỗ máy tuyến tính (chỉ có thể tạo ra các ranh giới thẳng tắp, không thể vẽ đường cong). Do đó, SVM nguyên bản sẽ bó tay trước các bài toán phi tuyến tính như XOR, giống hệt như Perceptron.

Tuy nhiên, SVM với Kernel (như RBF) lại giải quyết được bài toán XOR rất xuất sắc. Cách nó làm điều này rất đặc biệt: **Nó không giải quyết vấn đề bằng cách tự biến mình thành một cỗ máy vẽ đường cong, mà nó giải quyết bằng cách bẻ cong không gian chứa dữ liệu**.

Để hiểu rõ sự khác biệt tinh tế này, hãy hình dung bài toán XOR dưới dạng vật lý.

Bạn có một tờ giấy phẳng đặt trên bàn, đại diện cho không gian 2 chiều ban đầu. Trên tờ giấy có hai chấm màu đỏ (đèn sáng) nằm chéo nhau, và hai chấm màu xanh (đèn tắt) nằm ở hai góc chéo còn lại. Vì giới hạn tuyến tính, bạn chỉ được phép dùng một chiếc thước kẻ thẳng để chia tách chúng. Rõ ràng, không có cách nào đặt thước thẳng mà chia tách được hai nhóm màu này.

Thay vì cố gắng tìm cách uốn cong chiếc thước kẻ, thủ thuật Kernel (hàm ϕ) của SVM tiếp cận theo hướng khác. Hãy tưởng tượng tờ giấy chứa dữ liệu là một màng cao su đàn hồi. Hàm ϕ đóng vai trò như một tác động vật lý: nó kéo giãn và nhấc bổng trung tâm của màng cao su lên không trung, biến không gian từ 2 chiều (mặt phẳng) thành 3 chiều (độ cao).

Lúc này, các chấm dữ liệu thay đổi vị trí trong không gian mới. Giả sử sự biến dạng này đẩy hai chấm màu đỏ bay lên cao, trong khi hai chấm màu xanh vẫn nằm thấp bên dưới.

Khi dữ liệu đã nằm lơ lửng ở các độ cao khác nhau, SVM quay lại làm đúng công việc sở trường của nó: dùng một ranh giới thẳng tắp. Nó lấy một tấm bìa cứng phẳng (đại diện cho mặt phẳng quyết định 2D trong không gian 3D) và trượt ngang qua khoảng không nằm giữa nhóm chấm đỏ ở trên và nhóm chấm xanh ở dưới. Tấm bìa cứng này hoàn toàn phẳng, phân chia hoàn hảo hai nhóm dữ liệu, chứng tỏ bản chất của mô hình vẫn là tuyến tính (cộng dồn độc lập các yếu tố).

Tuy nhiên, khi bạn nhìn từ trên cao rọi thẳng xuống tờ giấy ban đầu, bóng của tấm bìa cứng in lên mặt giấy sẽ tạo thành một đường cong khép kín khoanh vùng hai chấm màu đỏ.

Đó là lý do tại sao tài liệu nhấn mạnh: "SVM vẫn là một mô hình tuyến tính, nó chỉ tuyến tính trong không gian ϕ ". Nó đã lách luật bằng cách mang dữ liệu đến một không gian nhiều chiều hơn, nơi dữ liệu rời rắm bỗng dưng có thể được cắt làm đôi bằng một mặt phẳng.

Điểm khác biệt quyết định giữa SVM (lựa chọn a) và Deep Learning (lựa chọn c) nằm ở cách tạo ra sự biến dạng không gian này.

Trong SVM, hàm biến dạng ϕ (Kernel) được cố định từ trước theo công thức toán học do con người chọn. Mô hình không biết cách tự tìm ra hướng kéo giãn tốt nhất, nó chỉ áp dụng máy móc một quy luật biến dạng không gian (ví dụ: điểm gần nhau thì dính với nhau) và hy vọng sau khi biến dạng, dữ liệu sẽ tách rời ra.

Trong khi đó, mạng Nơ-ron (Deep Learning) không sử dụng một hàm biến dạng cố định. Các tầng ẩn (hidden layers) chính là quá trình mô hình tự học cách nhào nặn, bóp méo, và kéo giãn không gian dữ liệu hết lần này đến lần khác, sao cho đến tầng cuối cùng, dữ liệu được sắp xếp gọn gàng nhất để có thể cắt dễ dàng bằng một đường thẳng. Mạng nơ-ron không chỉ học cách phân loại, mà nó học cả cách xây dựng hàm biến dạng không gian ϕ tối ưu nhất cho riêng bài toán đó

Gradient Descent

Definition

Khi chỉ có một biến duy nhất (một trọng số w), thứ tính ra gọi là **đạo hàm** (derivative), ký hiệu dL/dw .

Gradient là khi có từ hai biến trở lên. Lúc đó không tính ra một con số, mà tính ra một danh sách các đạo hàm riêng phần, mỗi cái ứng với một biến. Ví dụ có hai tham số w và b :

$$\text{gradient} = [dL/dw, dL/db]$$

Vậy chính xác hơn: chain rule tính ra từng đạo hàm riêng phần một, gradient là tập hợp tất cả các đạo hàm riêng phần đó gộp lại

How it actually work

Chain rule giúp ta thấy được như ở ví dụ trên tác động của từng cái **riêng lẻ** b và w tới L (ở đây là loss function) để khi dùng gradient descent ta biết được mỗi lần tăng giảm 2 số bao nhiêu thì có được loss function bao nhiêu

Ko chắc lắm

Nhờ đâu 2 số random ban đầu được đưa vào ở b và w nó xa giá trị đúng thì sao ví dụ b tối ưu là 2 còn w tối ưu là 5, random cả 2 đều là 1 (thực tế nó khác nhau) thì b sẽ chạm tối ưu trước w do cùng learning rate. Chỗ này khá hay là gần giá trị tối ưu thì nó sẽ giảm dần nên b sẽ giảm dần còn w giảm nhanh hơn

- Với 1 cái có vẽ hay là cả mạng neural network nó là tuyến tính nhưng qua activation function nó mới thành phi tuyến tính

THE CHAIN RULE !!!

Giống đặt u = hàm của x để tiện đạo hàm lúc cấp nhưng

$$\text{hunger} = \text{time}^2 + 1/2$$

$$\text{wantToEat} = \text{sqrt}(\text{hunger})$$

$$\text{wantToEat} = \sqrt{\text{hunger}} = \sqrt{\text{time}^2 + 1/2}$$

Gọi $\text{time}^2 + 1/2$ là u

$$\frac{d\text{WantToEat}}{d\text{Time}} = \frac{d\text{WantToEat}}{dU} \times \frac{dU}{d\text{Time}} = \frac{1}{2 \times \sqrt{u}} \times 2 \times \text{Time}$$

Thế $u = \text{time}^2 + 1/2$ vào là xong

Lí do cần biết nó

- Residual là thước đo mức độ sai
- Chain rule là cách tính xem mỗi tham số góp phần vào cái sai đó bao nhiêu, dù nó nằm ở xa qua nhiều bước trung gian
- Gradient descent dùng con số đó để biết nên chỉnh tham số theo hướng nào, chỉnh từng chút một cho đến khi sai số nhỏ nhất có thể

Loss Function

Perspective

Mô hình không xuất ra nhãn, nó xuất ra niềm tin

Đây là chỗ đổi góc nhìn quan trọng nhất, và nó nối thẳng với perceptron.

Perceptron xuất ra một phán quyết cứng: lớp A hoặc lớp B. Không có sắc thái. Một bài báo mà nó chắc chắn 99% và một bài nó phân vân 51-49 đều cho ra cùng một đầu ra.

Mô hình xác suất thì xuất ra **một phân bố** trên toàn bộ các lớp. Với bộ tin tức 13 chủ đề của bạn, đầu ra là 13 con số dương cộng lại bằng 1:

thể thao 0.70 · kinh tế 0.15 · chính trị 0.08 · ...

Đây chính là việc mà softmax làm ở tầng cuối (mục 6.2.2.3). Và ngay khi mô hình phát biểu dưới dạng xác suất, ta có một tiêu chuẩn khách quan để chấm điểm nó — chứ không còn phải tự chế ra công thức nữa.

Maximum likelihood

Nguyên tắc phát biểu bằng một câu:

Chọn bộ tham số làm cho dữ liệu bạn thực sự quan sát được trở nên ít bất ngờ nhất.

Bạn có 184 nghìn bài báo, mỗi bài có nhãn thật. Với mỗi bài, mô hình gán một xác suất cho nhãn thật đó. Một mô hình tốt là mô hình gán xác suất cao cho những nhãn đã thực sự xảy ra.

Vậy điểm số của toàn bộ mô hình = tích của 184 nghìn xác suất ấy. Càng lớn càng tốt.

Nhưng nhân 184 nghìn số nhỏ hơn 1 với nhau thì kết quả nhỏ tới mức máy tính làm tròn thành 0 ngay lập tức. Nên ta lấy **logarit**. Log biến tích thành tổng, và vì log là hàm tăng nghiêm ngặt nên **bộ tham số làm tích lớn nhất cũng chính là bộ làm tổng log lớn nhất** — không mất mát gì về mặt lời giải, chỉ được lợi về mặt tính toán.

Cuối cùng, các thuật toán tối ưu theo quy ước đều đi **giảm** chứ không tăng. Nên ta đổi dấu.

Kết quả là **negative log-likelihood**: tổng của các log xác suất, đổi dấu, chia cho số mẫu. Đó là hàm mất mát. Không ai nghĩ ra nó cả — nó rơi ra từ việc phát biểu bài toán cho tử tế.

Hoá ra nó chính là cross-entropy

Đây là điểm mà mục 6.2.1 nhấn mạnh và rất dễ đọc lướt.

Cross-entropy (giới thiệu ở 3.13, tr. 71) là một đại lượng từ lý thuyết thông tin, đo **độ chênh giữa hai phân bố xác suất**: phân bố thật của dữ liệu, và phân bố mà mô hình tin. Chênh càng lớn, cross-entropy càng cao.

Và hoá ra negative log-likelihood với cross-entropy là **cùng một thứ, viết theo hai ngôn ngữ khác nhau**. Sách nói thẳng: bất kỳ hàm mất mát nào là negative log-likelihood đều là một cross-entropy giữa phân bố thực nghiệm của tập huấn luyện và phân bố của mô hình

Probability (Xác suất) - Bài toán suy xuôi

Hãy hình dung khái niệm này qua lăng kính của việc thiết kế một hệ thống xử lý logic. Probability và Likelihood thực chất là hai cách gọi khi truyền tham số vào cùng một hàm toán học, điểm khác biệt duy nhất là bạn chọn xem biến nào là hằng số (đã biết) và biến nào là dữ liệu động (chưa biết).

Ở góc nhìn này, hệ thống của bạn đã được cấu hình xong. Các quy luật, tham số hay "đường cong phân bố" đã được chốt cứng. Dữ liệu thực tế chưa xảy ra.

Bạn đặt câu hỏi: **Hệ thống này sẽ sinh ra dữ liệu gì?**

- **Tình huống:** Bạn cầm trên tay một đồng xu hoàn hảo được đúc ở nhà máy. Quy luật (tham số) ở đây là tỷ lệ sấp ngửa chắc chắn bằng nhau.
- **Hành động:** Bạn tung đồng xu đó.
- **Kết quả:** Xác suất (Probability) để nó rơi vào mặt ngửa là 0.5. Bạn lấy cấu hình hệ thống làm chuẩn để dự đoán dữ liệu đầu ra ở tương lai.

Likelihood (Tính hợp lý) - Bài toán suy ngược

Ở góc nhìn này, hệ thống là một cấu hình chưa rõ ràng. Ngược lại, dữ liệu thực tế đã xảy ra rồi và được lưu cứng lại thành một log file. Bạn không thể dự đoán tương lai nữa vì kết quả đã nằm sờ sờ trước mặt.

Bạn đặt câu hỏi: **Cấu hình nào của hệ thống giải thích tốt nhất cho dữ liệu tôi đang thấy?**

- **Tình huống:** Bạn nhặt được một đồng xu lạ, tung thử 10 lần và cả 10 lần đều ra mặt ngửa (dữ liệu cố định).
- **Hành động:** Bạn đưa ra hai giả thuyết về cấu hình của đồng xu. Giả thuyết A: Nó là đồng xu bình thường. Giả thuyết B: Nó là đồng xu lỗi, hai mặt đều là ngửa.
- **Kết quả:** Dữ liệu "10 lần ngửa" khiến giả thuyết B trở nên cực kỳ hợp lý, trong khi làm giả thuyết A trở nên rất khó tin. Ta kết luận Likelihood của giả thuyết B cao hơn. Bạn dùng dữ liệu thực tế làm chuẩn để "chấm điểm" các cấu hình hệ thống khác nhau.

Mối liên hệ với Loss Function trong hệ thống phân loại

Toàn bộ quá trình huấn luyện mô hình thực chất là đi giải quyết bài toán suy ngược (Likelihood) này

Yếu tố	Vai trò trong Machine Learning
Dữ liệu (Cố định)	Tập dữ liệu training mà bạn có sẵn (đã biết rõ input và nhãn đúng).
Tham số (Thay đổi)	Bộ trọng số (weights) bên trong mạng nơ-ron.
Mục tiêu (Tối ưu)	Cập nhật các trọng số liên tục để nhãn đúng đạt độ hợp lý (likelihood) cao nhất.

Khi thiết kế thuật toán **Maximum Likelihood**, mục tiêu là tìm ra bộ trọng số làm cho dữ liệu training trở nên hợp lý nhất. Nếu bạn có hàng triệu điểm dữ liệu, máy tính phải nhân hàng triệu con số nhỏ hơn 1 lại với nhau, dẫn đến lỗi tràn số (underflow).

Để giải quyết vấn đề kỹ thuật này, hệ thống lấy logarithm để biến phép nhân thành phép cộng, và đổi dấu để biến bài toán "tìm giá trị cao nhất" thành "tìm giá trị thấp nhất" cho dễ tính đạo hàm. Hàm số sau khi được biến đổi xong chính là **Cross-Entropy Loss**. Tối ưu hàm Loss thực chất chỉ là một thủ thuật toán học để tối đa hóa Likelihood.

Đáng xem

Ở trên căn bản là nguồn gốc ra đời còn về ứng dụng thực tế

- Ở phần naive bayes ta cũng đụng vấn đề về tích của probability $\rightarrow$ dần về 0 (rất nhỏ dẫn đến tràn)
- Xử lý tương tự cũng là tích của log

$$-\sum_{c=1}^M \text{Observed}_c \times \log(\text{Predicted}_c)$$

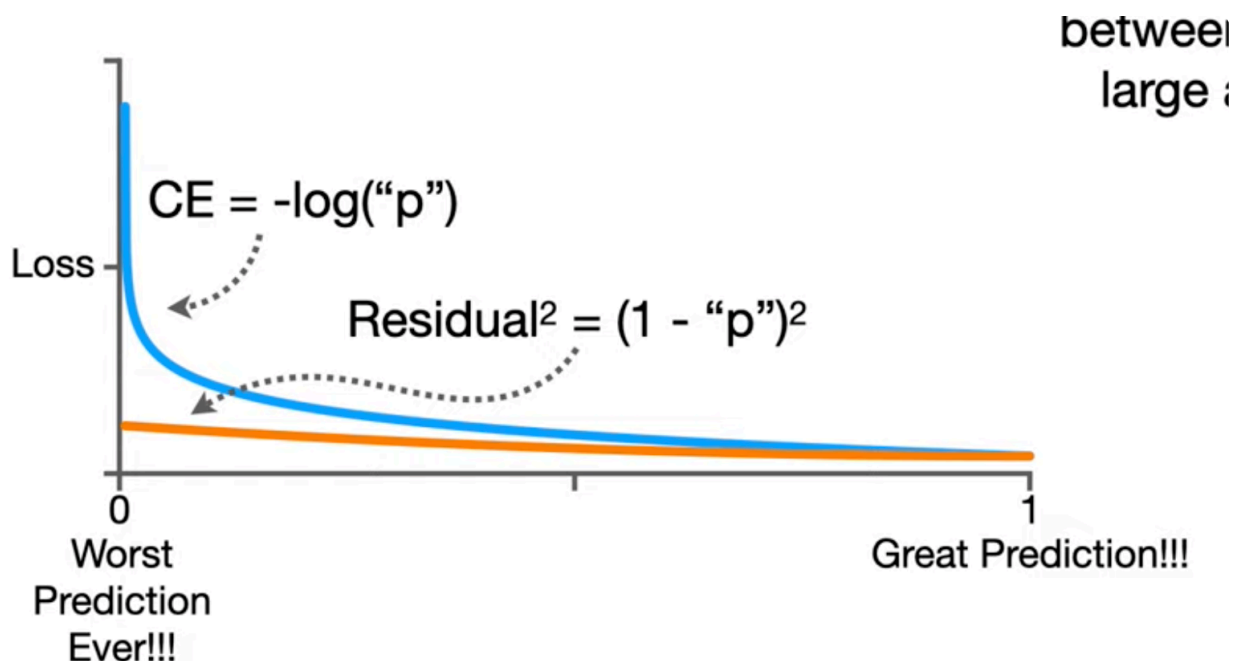
Ở trên là công thức tổng quát nhưng khi xét trong neural network ta muốn các mẫu khác = 0 nên chỉ cần 1 dạng đơn giản là

$$-\log(\text{Predicted}_c)$$

total error sẽ bằng total entropy

- Predicted_c ở đây là prob sau khi qua bước cuối là softmax

Lí do tại sao không phải SSR



Hình trên thể hiện khá rõ. Tốc độ thay đổi hay (slope) chính là đạo hàm, mà ta có thể slope của cross entropy căng hơn ssr nên khi bắt đầu bước nhảy cũng lớn hơn và ta tiến gần tới giá trị tối ưu nhanh hơn

Memory-based learning - Thứ bị bác bỏ

Ý tưởng đơn giản đến mức gần như phản trực giác:

Đừng học gì cả. Cứ nhớ hết. Khi gặp trường hợp mới, tìm những trường hợp cũ giống nó nhất rồi trả lời theo.

Không có trọng số, không có hidden layer, không có loss function, không có gradient. Toàn bộ "mô hình" chính là tập dữ liệu huấn luyện được lưu nguyên vẹn.

Argmax & Softmax

- argmax
 - Lớn nhất = 1
 - còn lại = 0
 - ⇒ Slope cả 2 line đều là 0 → không optimize được tí nào trong gradient descent
- softmax

$$\frac{e^i}{e^1 + e^2 + \dots + e^n}$$

- Nó sẽ ra 1 prob mà khi cộng tất cả lại sẽ bằng đúng 1

Hidden units và ReLU

Sách mở đầu bằng một lời thú nhận

Điều đầu tiên mục 6.3 nói ra, và đáng để bạn ghi nhớ vì nó hiếm trong sách giáo khoa: **việc chọn activation cho hidden layer là lĩnh vực đang nghiên cứu, chưa có nguyên lý lý thuyết dứt khoát nào chỉ đường.** Trong thực tế người ta chọn bằng thử nghiệm — đoán một cái, huấn luyện, đo trên tập validation.

Điều này khác hẳn phần loss function bạn vừa học. Ở đó, maximum likelihood cho ra câu trả lời **bắt buộc**: khai báo phân bố xong là loss tự rơi ra. Ở đây không có nguyên lý nào tương đương. Sách chỉ đưa ra một khuyến nghị mặc định: **dùng ReLU, trừ khi có lý do cụ thể để không dùng.**

Đừng đọc lướt sự khác biệt này. Nó cho bạn biết chỗ nào đáng bỏ công thử nghiệm và chỗ nào không.

ReLU là gì

Bạn đã gặp nó rồi, ở bài XOR: **cắt bỏ phần âm, giữ nguyên phần dương.**

Số vào là 3 thì ra 3. Số vào là 7 thì ra 7. Số vào là -2 thì ra 0. Số vào là -100 cũng ra 0.

Hết. Không có hàm mũ, không có phép chia, chỉ một phép so sánh với 0. Đây là hàm phi tuyến rẻ nhất mà người ta biết.

Vì sao nó thắng, dù trông thô sơ

Lý do một: nó gần như tuyến tính, và đó là một ưu điểm.

Đây là lập luận trung tâm của mục 6.3.1, và nó ngược trực giác. Mô hình tuyến tính dễ tối ưu — bạn luôn có hướng đi rõ ràng. ReLU giữ lại gần hết tính chất đó: ở nửa dương nó **đúng là** hàm đồng nhất, phẳng lì, không cong. Nó chỉ gãy đúng một chỗ tại 0.

Bài học rút ra: bạn không cần một hàm phi tuyến *mạnh*. Bạn chỉ cần **đủ phi tuyến để phá tính cộng dồn**, và càng gần tuyến tính thì càng dễ huấn luyện. Trong ví dụ XOR, một cú gãy duy nhất tại 0 đã đủ giải bài toán mà perceptron bó tay.

Lý do hai: gradient không bị co lại.

Đây mới là lý do khiến ReLU thay đổi cả ngành. Ở nửa dương, độ dốc của ReLU **đúng bằng 1**. Không lớn hơn, không nhỏ hơn.

So với sigmoid, hàm mà cả ngành dùng suốt thập niên 1990. Độ dốc lớn nhất của sigmoid chỉ là 0.25, và nó nhỏ hơn nhiều ở hai đầu. Khi tín hiệu học truyền ngược qua các tầng, các độ dốc **nhân với nhau**. Qua 10 tầng sigmoid, tín hiệu bị nhân với 0.25 mười lần — tức chia cho khoảng một triệu.

Kết quả: các tầng đầu gần như không nhận được tín hiệu học nào. Đó là **vanishing gradient**, và nó là lý do người ta không huấn luyện nổi mạng sâu trong suốt hai thập kỷ. ReLU nhân 1 với 1 với 1 — tín hiệu đi qua nguyên vẹn.

Sách nói thẳng ở mục 6.3.2: **không nên dùng sigmoid làm hidden unit**. Nó vẫn hợp lý ở tầng output (vì log của loss triệt tiêu hàm mũ, như bạn đã học), nhưng ở tầng ẩn thì không có gì cứu nó cả.

Hai chi tiết sách nêu mà người học hay bỏ qua

ReLU không khả vi tại 0 — và không ai bận tâm.

Về mặt toán, tại đúng điểm 0 hàm này gãy góc, không có đạo hàm. Nghe như một lỗi hổng nghiêm trọng cho một thuật toán dựa hoàn toàn vào đạo hàm.

Sách xử lý điểm này thẳng thắn: trong thực tế không thành vấn đề. Thứ nhất, xác suất một con số dấu phẩy động rơi đúng vào 0 gần như bằng không. Thứ hai, thư viện chỉ việc trả về một trong hai đạo hàm một phía — PyTorch trả 0. Thứ ba, và đây là lý do sâu

nhất: gradient descent trên mạng nơ-ron **không bao giờ tới được điểm cực tiểu chính xác**, nên việc đạo hàm hơi sai ở một điểm đơn lẻ là chuyện vặt.

Đây là một ví dụ tốt về khoảng cách giữa tính chặt chẽ toán học và cái thật sự quan trọng khi chạy.

Khởi tạo bias bằng số dương nhỏ.

Sách khuyến nghị đặt bias ban đầu là 0.1 thay vì 0, để các ReLU **đang hoạt động** ngay từ đầu và nhận được gradient. Chi tiết nhỏ, nhưng nó dẫn tới vấn đề tiếp theo.

Dying ReLU

Mặt trái của việc gradient bằng 0 ở nửa âm: nếu một unit bị đẩy vào vùng luôn cho ra 0 với **mọi** mẫu dữ liệu, thì nó không nhận gradient nữa. Không gradient thì không cập nhật. Không cập nhật thì mắc kẹt vĩnh viễn. Unit đó chết — chiếm bộ nhớ, tốn phép tính, đóng góp bằng không.

Thường xảy ra khi learning rate quá lớn, một bước cập nhật mạnh đẩy unit ra khỏi vùng sống.

Mục 6.3.1 giới thiệu các biến thể chữa việc này, tất cả cùng ý tưởng: **cho nửa âm một độ dốc nhỏ khác 0** để unit còn đường quay lại. Leaky ReLU dùng độ dốc cố định nhỏ, PReLU học luôn độ dốc đó. Sách cũng trình bày maxout, một hướng tổng quát hơn.

Thực tế: với một mạng nông như của bạn, ReLU thuần là đủ. Nếu thấy nhiều unit chết, cách chữa đầu tiên là **giảm learning rate**, không phải đổi activation.

Chuyên môn hoá ?

Không rõ đây có phải phần tựa tựa cái đóng băng trọng số không nhưng có 1 ý khá hay

Một unit cho ra 0 với **một bài báo cụ thể** là chuyện hoàn toàn bình thường và cần thiết. Ở bất kỳ thời điểm nào, khoảng một nửa số unit trong mạng đang ở trạng thái tắt. Sách

coi đây là một **ưu điểm**, gọi là kích hoạt thưa: mỗi unit chỉ phản ứng với một phần của không gian đầu vào, tức là nó **chuyên môn hoá**

Backpropagation

Check out the quest =)) to much to take note

References

[Machine Learning cơ bản: Perceptron và Nền tảng của Deep Learning](#)

[The Chain Rule, Clearly Explained!!!](#)

[Gradient Descent, Step-by-Step](#)

[Neural Networks Part 6: Cross Entropy](#)

[Maximum Likelihood, clearly explained!!!](#)

[Mạng nơ-ron nhân tạo Phần 5: ArgMax và SoftMax](#)

[Neural Networks Pt. 3: ReLU In Action!!!](#)

[Neural Networks Pt. 2: Backpropagation Main Ideas](#)

[Backpropagation Details Pt. 1: Optimizing 3 parameters simultaneously](#)

[Backpropagation Details Pt. 2: Going bonkers with The Chain Rule](#)

[Neural Networks Pt. 4: Multiple Inputs and Outputs](#)